

React Location Tracking UI Guide

This guide explains how to add **live location tracking (latitude & longitude)** with an interactive map in a React app. It covers three popular libraries: React Leaflet, Google Maps, and Mapbox. We also include setup notes for Android WebView so your hosted React app can request GPS inside a native Android wrapper.

1. React Leaflet (OpenStreetMap)

■ Free, no API key required. Good for simple tracking.

1. Install dependencies: `bash npm install react-leaflet leaflet` 2. Import CSS in index.js: `js import "leaflet/dist/leaflet.css";` 3. Example Code:

```
import { MapContainer, TileLayer, Marker, Popup } from "react-leaflet";
import { useState, useEffect } from "react";

export default function TrackingMap() {
  const [position, setPosition] = useState([0, 0]);

  useEffect(() => {
    navigator.geolocation.watchPosition((pos) => {
      setPosition([pos.coords.latitude, pos.coords.longitude]);
    });
  }, []);

  return (
    <MapContainer center={position} zoom={15} style={{ height: "100vh" }}>
      <TileLayer url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png" />
      <Marker position={position}>
        <Popup>You are here</Popup>
      </Marker>
    </MapContainer>
  );
}
```

2. Google Maps (@react-google-maps/api)

■ Professional look, needs a free Google Maps API key.

1. Get a Google Maps API key from: <https://console.cloud.google.com/> 2. Install package: `bash npm install @react-google-maps/api` 3. Example Code:

```
import { GoogleMap, Marker, useJsApiLoader } from "@react-google-maps/api";
import { useState, useEffect } from "react";

export default function GoogleTrackingMap() {
  const { isLoaded } = useJsApiLoader({
    googleMapsApiKey: "YOUR_API_KEY",
  });

  const [position, setPosition] = useState({ lat: 0, lng: 0 });

  useEffect(() => {
    navigator.geolocation.watchPosition((pos) => {
      setPosition({ lat: pos.coords.latitude, lng: pos.coords.longitude });
    });
  });
}
```

```

}, []);

if (!isLoading) return <p>Loading...</p>;

return (
  <GoogleMap
    center={position}
    zoom={15}
    mapContainerStyle={{ height: "100vh", width: "100%" }}
  >
    <Marker position={position} />
  </GoogleMap>
);
}

```

3. Mapbox (react-map-gl)

■ Uber-style smooth animations, needs a free Mapbox access token.

1. Sign up at <https://mapbox.com> and get an access token. 2. Install package: `bash npm install react-map-gl` 3. Example Code:

```

import Map, { Marker } from "react-map-gl";
import { useState, useEffect } from "react";

export default function MapboxTracking() {
  const [pos, setPos] = useState({ lat: 0, lng: 0 });

  useEffect(() => {
    navigator.geolocation.watchPosition((p) => {
      setPos({ lat: p.coords.latitude, lng: p.coords.longitude });
    });
  }, []);

  return (
    <Map
      mapboxAccessToken="YOUR_MAPBOX_TOKEN"
      initialState={{ longitude: pos.lng, latitude: pos.lat, zoom: 14 }}
      style={{ width: "100%", height: "100vh" }}
      mapStyle="mapbox://styles/mapbox/streets-v11"
    >
      <Marker latitude={pos.lat} longitude={pos.lng} />
    </Map>
  );
}

```

4. Android WebView Wrapper Setup

If your React app is wrapped in a native Android WebView, you must enable geolocation permissions. AndroidManifest.xml: `xml` WebView Java setup: `java`

```

webView.getSettings().setJavaScriptEnabled(true);
webView.getSettings().setGeolocationEnabled(true); webView.setWebChromeClient(new
WebChromeClient() { @Override public void onGeolocationPermissionsShowPrompt( String origin,
GeolocationPermissions.Callback callback) { callback.invoke(origin, true, false); } });

```

5. Comparison Table

Library	Pros	Cons
React Leaflet	Free, lightweight, no API key	Basic look, fewer features
Google Maps	Professional, familiar UI, reliable	Needs API key, billing setup after free tier
Mapbox	Uber-style UI, smooth animations	Needs token, slightly heavier